

Threads

Introduction

CS232 Operating Systems, Fall 2019

Taj Muhammad Khan

Threads

- So far ...
 - There's only one point of execution in a given process (PC, stack, etc.)
 - i.e. one tread!
- In a multi-threaded program, there are ...
 - Multiple points of execution!
 - i.e. multiple PCs, multiple stacks, etc.
 - Threads *share* the *same address space !!!*

Threads vs Processes

- Threads are like processes:
 - They can execute independently
 - Each thread has a separate PC and set of registers (context) while executing
 - If two threads T1 & T2 are running on a single processor then:
 - Only one can run at any given time
 - Switching from one thread to other requires a context switch
 - Each thread has its own stack (thread local storage)
- Threads are different than processes:
 - Threads share the same address space
 - Context switch b/w threads results in switching of stacks but not of page tables!
 - A single process can have multiple threads
 - Thread states are saved in Thread Control Blocks (TCBs) instead of PCBs

Threads vs Processes

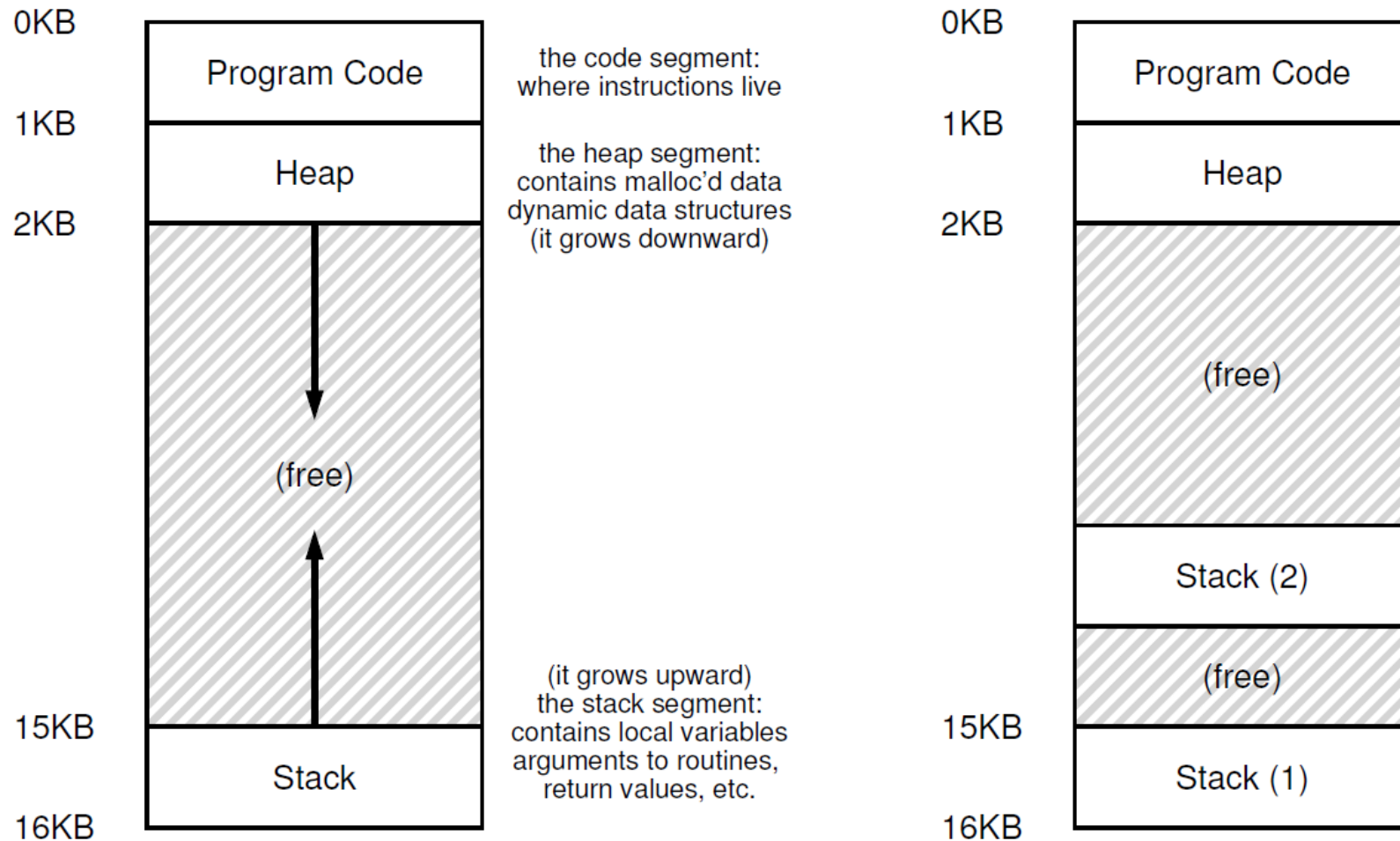


Figure 26.1: Single-Threaded And Multi-Threaded Address Spaces

Why use threads?

- Parallelism
 - Better exploit multi-core CPUs
- Blocking I/O
 - When doing a blocking I/O operation, only one thread gets blocked!
 - Overlap I/O with other activities within a process
- Same could be achieved via multiple processes but ... ?
- Threads are extensively used in Web servers, DBMSs, etc.

Threads – example

- How many threads are run ??

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4  #include "common.h"
5  #include "common_threads.h"
6
7  void *mythread(void *arg) {
8      printf("%s\n", (char *) arg);
9      return NULL;
10 }
11
12 int
13 main(int argc, char *argv[]) {
14     pthread_t p1, p2;
15     int rc;
16     printf("main: begin\n");
17     Pthread_create(&p1, NULL, mythread, "A");
18     Pthread_create(&p2, NULL, mythread, "B");
19     // join waits for the threads to finish
20     Pthread_join(p1, NULL);
21     Pthread_join(p2, NULL);
22     printf("main: end\n");
23     return 0;
24 }
```

Threads – example: scheduling

main	Thread 1	Thread2
starts running		
prints “main: begin”		
creates Thread 1		
creates Thread 2		
waits for T1	runs	
	prints “A”	
	returns	
waits for T2		runs
		prints “B”
		returns
prints “main: end”		

Figure 26.3: Thread Trace (1)

Threads – sharing data

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "common.h"
4  #include "common_threads.h"
5
6  static volatile int counter = 0;
7
8  //
9  // mythread()
10 //
11 // Simply adds 1 to counter repeatedly, in a loop
12 // No, this is not how you would add 10,000,000 to
13 // a counter, but it shows the problem nicely.
14 //
15 void *mythread(void *arg) {
16     printf("%s: begin\n", (char *) arg);
17     int i;
18     for (i = 0; i < 1e7; i++) {
19         counter = counter + 1;
20     }
21     printf("%s: done\n", (char *) arg);
22     return NULL;
23 }
```

Threads – sharing data ... contd.

```
//  
// main()  
//  
// Just launches two threads (pthread_create)  
// and then waits for them (pthread_join)  
//  
int main(int argc, char *argv[]) {  
    pthread_t p1, p2;  
    printf("main: begin (counter = %d)\n", counter);  
    Pthread_create(&p1, NULL, mythread, "A");  
    Pthread_create(&p2, NULL, mythread, "B");  
  
    // join waits for the threads to finish  
    Pthread_join(p1, NULL);  
    Pthread_join(p2, NULL);  
    printf("main: done with both (counter = %d)\n", counter);  
    return 0;  
}
```

Threads – sharing data ... contd.2

- Better run code here ...

Threads – sharing data ... contd.3

- Why the anomalous result?
- The code for `counter = counter + 1`; translated as:

```
100  mov     0x8049a1c, %eax
105  add     $0x1, %eax
108  mov     %eax, 0x8049a1c
```

Threads – sharing data ... contd.4

OS	Thread 1	Thread 2	(after instruction)		
			PC	%eax	counter
	<i>before critical section</i>		100	0	50
	mov 0x8049a1c, %eax		105	50	50
	add \$0x1, %eax		108	51	50
interrupt					
	<i>save T1's state</i>				
	<i>restore T2's state</i>		100	0	50
		mov 0x8049a1c, %eax	105	50	50
		add \$0x1, %eax	108	51	50
		mov %eax, 0x8049a1c	113	51	51
interrupt					
	<i>save T2's state</i>				
	<i>restore T1's state</i>		108	51	51
	mov %eax, 0x8049a1c		113	51	51

Figure 26.7: The Problem: Up Close and Personal

Race conditions

- The result of a program is **indeterminate**
 - It depends on the sequence in which the instructions are scheduled!
- **Critical section:**
 - A portion of code which, if executed by multiple threads, can result in race condition
 - Usually accesses a shared resource
 - Should not be executed **concurrently** by more than one thread (**mutual exclusion**)

Atomicity

- Mutual exclusion can be achieved if critical sections are executed **atomically**.
- Atomic operations are executed as “all or nothing”
 - i.e. they cannot be interrupted half way.
 - Databases have similar concepts – transactions.

Threads: synchronization

- Sometimes threads need to wait for each other.

Operating System – the first concurrent program

- Multiple processes run concurrently
 - Let's say two processes try to append to a file
 - Possible problems?
 - Interrupts!!
-
- All OS data structures are shared b/w processes
 - Codes that update these shared data structures are critical sections